go through elements in a priority queue once, and the elements already removed from the list won't matter for the ordering anymore.

If, instead, we need to keep track of the ordering of elements and possibly be able to go through them more than once (like a list of objects to render on a web page), priority queues might not be the best choice. Moreover, there are other kind of operations that we might need to perform, for example efficiently retrieve the minimum or maximum element of our collection, or access the `i`-th element (without removing all elements before it), or yet finding the predecessor or successor of an element in our ordering.

In appendix C we discuss how trees are the best compromise when we care about all these operations, in addition to insertion and removal: if a tree is balanced, we can perform any of these actions in logarithmic time.

The issue is that trees in general and binary trees in particular are not guaranteed to be balanced: figure 2.1 shows how, depending on the order of insertion, we might have very balanced or very skewed trees.

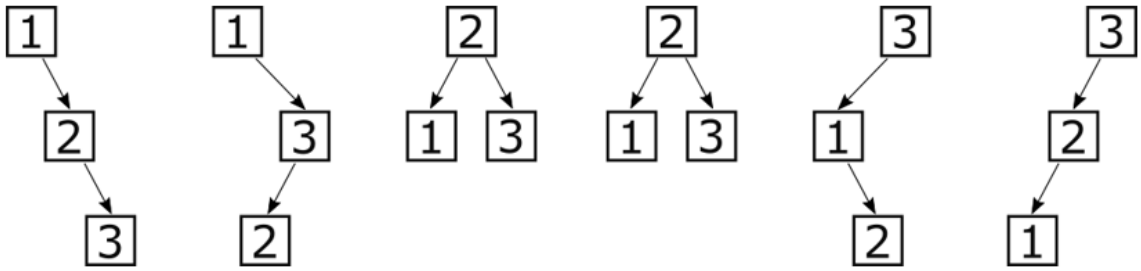


Figure 3.1 All possible layouts for *BST*s of size 3. The layout depends on the order in which elements are inserted. Notice how 2 of the layouts are identical: for the insertion sequences `[2, 1, 3]` and `[2, 3, 1]` we get the same final result.